

Trabalho Prático RISC-V

Lucas dos Anjos Camelo¹, João Vitor Soares Henriques²

¹ICEA – Universidade Federal de Ouro Preto (UFOP)

²Departamento de Engenharia da Computação DCEA
Universidade Federal de Ouro Preto (ICEA) – João Monlevade, MG – Brazil

joao.henriques@aluno.ufop.edu.br, lucas.camelo@aluno.ufop.edu.br

Abstract. *This paper presents the design and implementation of a 32-bit processor based on the RV32E specification of the RISC-V instruction set architecture (ISA). The RV32E is a variant optimized for embedded systems, featuring a reduced register file of 16 registers, which leads to a simpler design, smaller silicon area, and lower power consumption. The processor is implemented in Verilog HDL using a five-stage pipeline architecture (Fetch, Decode, Execute, Memory, and Write-back) to maximize instruction throughput. The project scope covers the implementation of an essential subset of instructions to validate the processor's basic functionality, serving as a case study for low-cost CPU design.*

Resumo. *Este trabalho apresenta o projeto e a implementação de um processador de 32 bits baseado na especificação RV32E do conjunto de instruções RISC-V. O RV32E é uma variante otimizada para sistemas embarcados, caracterizada por um banco de registradores reduzido a 16, o que resulta em um design mais simples, menor área de silício e consumo de energia reduzido. O processador foi implementado em Verilog HDL utilizando uma arquitetura de pipeline de cinco estágios (Busca, Decodificação, Execução, Memória e Escrita) para maximizar a vazão de instruções. O escopo do projeto abrange a implementação de um subconjunto essencial de instruções para validar a funcionalidade básica do processador, servindo como um estudo de caso para o design de CPUs de baixo custo.*

1. Introdução

Este documento descreve o projeto e a implementação de um processador de 32 bits que utiliza a arquitetura de conjunto de instruções (ISA) RISC-V, focando na especificação de inteiros base RV32E [1]. A arquitetura RISC-V é notável por seu modelo aberto e modular, que facilita a criação de processadores personalizados para múltiplas aplicações.

O RV32E é uma extensão do conjunto de instruções base de inteiros (RV32I), otimizada para sistemas embarcados e microcontroladores de baixo custo. Sua principal característica é a redução do banco de registradores de 32 para 16 (x0 a x15), o que simplifica o design do hardware, diminui a área de silício e reduz o consumo de energia, mantendo a compatibilidade com o ecossistema de software do RISC-V.

O objetivo deste projeto é desenvolver um processador com um pipeline de 5 estágios (Busca, Decodificação, Execução, Memória e Escrita), implementando um subconjunto essencial de instruções do RV32E para garantir a funcionalidade básica. A implementação será realizada em Verilog HDL.

Esta documentação servirá como um guia completo, abordando desde a análise da arquitetura e do conjunto de instruções até a descrição detalhada de cada módulo de hardware, sua lógica de funcionamento e a integração final no processador.

2. Desenvolvimento

Nesta seção, descreveremos a arquitetura do processador e detalharemos a implementação de cada um dos seus componentes fundamentais. A estrutura do projeto foi organizada de forma modular para facilitar o desenvolvimento, a simulação e a verificação de cada parte do sistema.

2.1. Arquitetura do Processador

A arquitetura escolhida para este processador é um pipeline clássico de 5 estágios. Esta abordagem permite a sobreposição da execução de múltiplas instruções, aumentando significativamente a vazão (throughput) do processador em comparação com uma implementação de ciclo único. Os estágios do pipeline são descritos a seguir:

- **IF (Instruction Fetch):** Busca da próxima instrução na memória.
- **ID (Instruction Decode):** Decodificação da instrução, leitura dos operandos do banco de registradores e geração de imediatos.
- **EX (Execute):** Execução da operação na Unidade Lógica e Aritmética (ULA) e cálculo de endereços de desvio.
- **MEM (Memory Access):** Acesso à memória de dados para operações de leitura (load) ou escrita (store).
- **WB (Write Back):** Escrita do resultado de volta ao banco de registradores.

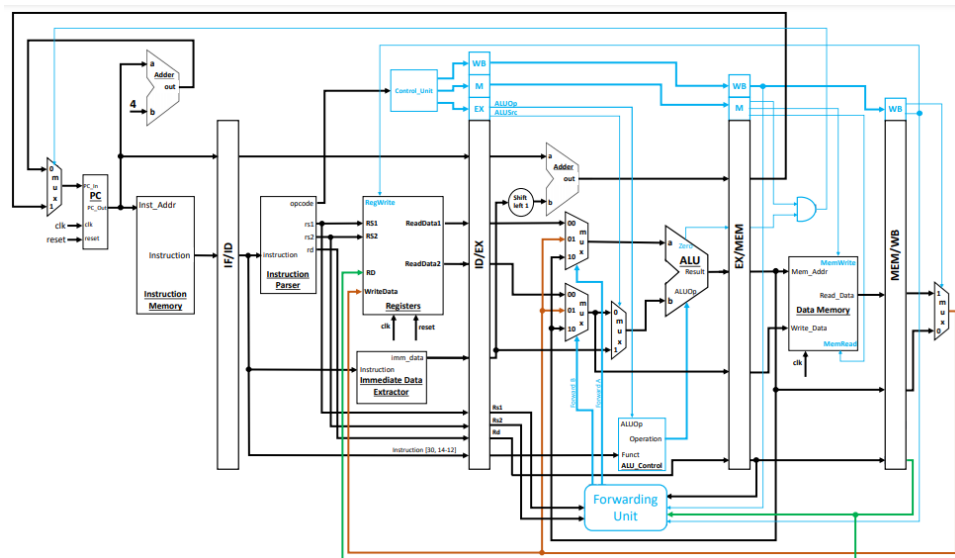


Figure 1. Arquitetura de pipeline de cinco estágios (IF, ID, EX, MEM, WB) utilizada no processador RV32E.

Como mostrado na Figura 1, o processador segue o modelo clássico de cinco estágios, permitindo que várias instruções sejam processadas simultaneamente em diferentes fases do ciclo de execução.

2.2. Componentes do Caminho de Dados (Datapath)

O caminho de dados (datapath) é composto pelos blocos funcionais responsáveis por armazenar, manipular e transferir os dados ao longo do pipeline. Estes componentes são a base sobre a qual os estágios do processador são construídos.

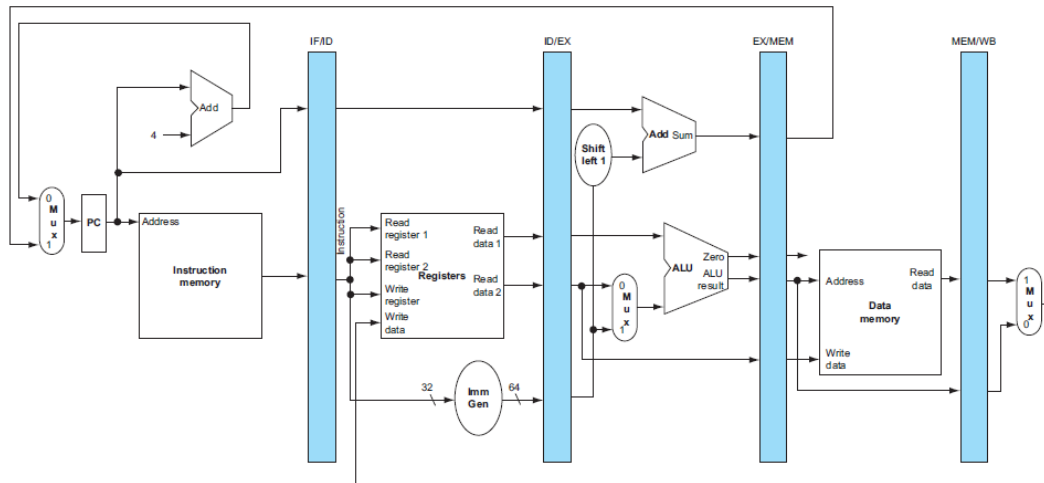


Figure 2. Datapath do processador RV32E implementado, destacando blocos funcionais, caminhos de dados e sinais de controle.

2.2.1. Unidade Lógica e Aritmética (ULA)

A Unidade Lógica e Aritmética (ULA), ou ALU (Arithmetic Logic Unit), é o componente central do estágio de Execução. Trata-se de um circuito puramente combinacional que realiza as operações matemáticas e lógicas definidas pelo conjunto de instruções. A operação específica a ser executada é selecionada por um sinal de controle (ALUControl) proveniente da Unidade de Controle.

Analisando o código-fonte da ULA apresentado na Figura 3, podemos observar sua implementação em *Verilog*. O módulo, nomeado `alu`, possui duas entradas de dados de 32 bits (`a` e `b`), uma entrada de controle de 4 bits (`ALUOp`) que determina a operação a ser realizada, uma saída de 32 bits para o resultado (`Result`) e uma saída de 1 bit (`Zero`).

O comportamento da ULA é definido dentro de um bloco `always @(*)`, que caracteriza o circuito como combinacional, ou seja, suas saídas reagem instantaneamente a qualquer mudança nas entradas. A seleção da operação é implementada através de uma estrutura `case`, que avalia o valor do sinal `ALUOp`. Cada valor possível para `ALUOp` corresponde a uma operação específica, conforme detalhado a seguir:

- **ALU_ADD e ALU_SUB:** Realizam as operações aritméticas fundamentais de adição ($\text{Result} = a + b$) e subtração ($\text{Result} = a - b$).
- **ALU_SLL:** Executa um deslocamento lógico para a esquerda (*shift left logical*), onde os bits da entrada `a` são deslocados para a esquerda por um número de posições determinado pelos 5 bits menos significativos da entrada `b` ($\text{Result} = a \ll b[4:0]$).

```

1  `timescale 1ns / 1ps
2  `include "alu_defines.vh"
3
4  module alu (
5      input wire [31:0] a,
6      input wire [31:0] b,
7      input wire [3:0] ALUOp,
8      output reg [31:0] Result,
9      output wire Zero
10 );
11
12 always @(*) begin
13     case (ALUOp)
14         `ALU_ADD: Result = a + b;
15         `ALU_SUB: Result = a - b;
16         `ALU_SLL: Result = a << b[4:0];
17         `ALU_SLT: Result = ($signed(a) < $signed(b)) ? 32'd1 : 32'd0;
18         `ALU_SLTU: Result = (a < b) ? 32'd1 : 32'd0;
19         `ALU_XOR: Result = a ^ b;
20         `ALU_SRL: Result = a >> b[4:0];
21         `ALU_SRA: Result = $signed(a) >>> b[4:0];
22         `ALU_OR: Result = a | b;
23         `ALU_AND: Result = a & b;
24         `ALU_LUI: Result = b;
25         default: Result = 32'd0;
26     endcase
27 end
28
29 assign Zero = (Result == 32'd0);
30
31 endmodule

```

Figure 3. Código fonte da ULA

- **ALU_SLT**: Implementa a operação *set on less than* para números com sinal. A saída `Result` será 1 se o valor de `a` (interpretado como número com sinal) for menor que o valor de `b` (também com sinal), e 0 caso contrário.
- **ALU_SLTU**: Similar à anterior, mas realiza a comparação *set on less than* para números sem sinal.
- **ALU_XOR, ALU_OR, ALU_AND**: Executam as operações lógicas bit a bit XOR (^), OR (|) e AND (&), respectivamente, entre as entradas `a` e `b`.
- **ALU_SRL e ALU_SRA**: Realizam operações de deslocamento para a direita. **ALU_SRL** (*shift right logical*) insere zeros à esquerda, enquanto **ALU_SRA** (*shift right arithmetic*) mantém o bit de sinal, o que é crucial para operações com números negativos.

Além do resultado principal, a ULA também gera o sinal `Zero`. Este sinal é ativado (1) sempre que o resultado (`Result`) da operação for igual a zero (`Result == 32'd0`). Essa saída é fundamental para a implementação de desvios condicionais no processador, como as instruções do tipo `branch` (e.g., `beq` – *branch on equal*), que precisam saber se o resultado de uma comparação foi zero.

2.2.2. Banco de Registradores

O banco de registradores (`Register File`) é uma memória interna de alta velocidade que armazena os dados de trabalho do processador. Para a especificação RV32E, ele é composto por 16 registradores de 32 bits (de `x0` a `x15`). O registrador `x0` é especial, pois seu valor é sempre zero, uma característica fundamental da arquitetura RISC-V.

O módulo possui duas portas de leitura paralelas e assíncronas (para `rs1` e `rs2`) e uma porta de escrita síncrona (para `rd`). A escrita ocorre apenas na borda de subida do clock e quando o sinal de controle `RegWrite` está ativo.

Analisando o código-fonte da Figura 4, vemos como essas características são implementadas:

1. **Estrutura e Parametrização:** O módulo `register_file` é projetado de forma genérica, utilizando parâmetros `WIDTH = 32` e `DEPTH = 32`. Isso define uma estrutura padrão de 32 registradores de 32 bits, compatível com a arquitetura RV32I. Para o RV32E, ele seria instanciado com uma profundidade de 16. O armazenamento principal é realizado pelo vetor de registradores:

```
reg [WIDTH-1:0] regs [0:DEPTH-1];
```

2. **Portas de Leitura Assíncronas:** As duas portas de leitura são implementadas com as instruções `assign` nas linhas 40 e 41. Por serem atribuições contínuas, criam um circuito puramente combinacional. Assim, sempre que os endereços `rs1` ou `rs2` mudam, os dados correspondentes são disponibilizados imediatamente nas saídas `read_data_1` e `read_data_2`, sem necessidade de esperar um pulso de clock.
3. **Porta de Escrita Síncrona:** A lógica de escrita e reset está contida no bloco:

```
always @(posedge clk or posedge rst)
```

```

1  `timescale 1ns / 1ps
2
3  module register_file #(
4      parameter WIDTH = 32,
5      parameter DEPTH = 32
6  )(
7      input wire          clk,
8      input wire          rst, // Reset assíncrono (1 = reset)
9
10     input wire [4:0]     rs1,
11     input wire [4:0]     rs2,
12     input wire [4:0]     rd,
13     input wire [WIDTH-1:0] wd,
14     input wire           rw,
15
16     output wire [WIDTH-1:0] read_data_1,
17     output wire [WIDTH-1:0] read_data_2,
18
19     input wire [$clog2(DEPTH)-1:0] debug_read_index,
20     output wire [WIDTH-1:0] debug_data_out
21 );
22
23     reg [WIDTH-1:0] regs [0:DEPTH-1];
24     integer i;
25
26     // Reset assíncrono + escrita síncrona
27     always @(posedge clk or posedge rst) begin
28         if (rst) begin
29             // Zera todos os registradores exceto x0 (que é hardwired para 0)
30             for (i = 1; i < DEPTH; i = i + 1) begin
31                 regs[i] <= 0;
32             end
33         end
34         else if (rw && rd != 5'd0) begin
35             regs[rd] <= wd;
36         end
37     end
38
39     // Leituras assíncronas
40     assign read_data_1 = (rs1 == 5'd0) ? 0 : regs[rs1]; // x0 é sempre 0
41     assign read_data_2 = (rs2 == 5'd0) ? 0 : regs[rs2]; // x0 é sempre 0
42     assign debug_data_out = (debug_read_index == 5'd0) ? 0 : regs[debug_read_index];
43
44 endmodule

```

Figure 4. Código fonte do banco de registradores

A sensibilidade à `posedge clk` garante que qualquer escrita no banco de registradores só aconteça exatamente na borda de subida do clock, tornando a operação síncrona.

4. **Implementação do Registrador x0 (Zero):** O código garante que o registrador `x0` seja sempre zero de três formas:
 - (a) **Na Escrita:** A condição `if (rw && rd != 5'd0)` impede qualquer operação de escrita caso o registrador de destino (`rd`) seja o de endereço 0.
 - (b) **Na Leitura:** As linhas 40 e 41 usam um operador ternário para verificar se o endereço de leitura (`rs1` ou `rs2`) é 0. Nesse caso, o valor 0 é retornado diretamente, em vez de ler `regs[0]`.
 - (c) **No Reset:** O laço `for` na lógica de reset inicia em `i = 1`, zerando todos os registradores exceto `regs[0]`.
5. **Reset Assíncrono:** A lista de sensibilidade do bloco `always` inclui `posedge rst`, caracterizando um reset assíncrono. Se o sinal `rst` for ativado, o banco de registradores entra no estado de reset imediatamente, zerando os registradores de `x1` a `x15` (ou `x31`), independentemente do clock.

2.2.3. PC Generator

O PC Generator é o circuito responsável por selecionar o endereço da próxima instrução que será enviada para o estágio de Busca (Fetch). Ele garante o fluxo de execução correto do programa, seja de forma sequencial ou através de desvios e saltos.

```
1  `timescale 1ns / 1ps
2
3  module pc_generator (
4      input wire      clk,
5      input wire      rst,
6      input wire [31:0] pc_in,    // Vem do MUX (PC + 4, branch, jump...)
7
8      output reg [31:0] pc_out    // Vai para instruction_memory e adder
9  );
10
11     always @(posedge clk or posedge rst) begin
12         if (rst)
13             pc_out <= 32'b0;
14         else
15             pc_out <= pc_in;
16     end
17
18 endmodule
```

Figure 5. Código fonte do PC Generator

Este sistema é composto por duas partes principais: uma lógica de seleção (geralmente um multiplexador) e o registrador do Contador de Programa (PC) propriamente dito. O código-fonte apresentado na imagem 5 implementa este último componente: o registrador PC.

Analisando o módulo `pc_generator`, podemos observar o seu comportamento:

- **Armazenamento Síncrono:** O módulo é, em sua essência, um registrador de 32 bits. Sua função é armazenar o endereço da instrução a ser buscada (`pc_out`). A atualização desse valor é síncrona, ocorrendo apenas na borda de subida do clock (`posedge clk`), conforme definido no bloco `always`.
- **Entrada do Próximo PC:** A entrada `pc_in` fornece o endereço que será carregado no PC no próximo ciclo de clock. Como indicado no comentário do código (`// Vem do MUX (PC + 4, branch, jump...)`), este sinal é o resultado da lógica de seleção que acontece externamente ao módulo. É nesse multiplexador que a decisão sobre qual será o próximo endereço (seja o sequencial `PC+4` ou um endereço de desvio/salto) é tomada.
- **Reset Assíncrono:** O registrador possui uma lógica de reset assíncrono. Se o sinal `rst` for ativado (1), o PC é imediatamente forçado para o valor 0 (`pc_out <= 32'b0;`). Isso garante que, ao ligar ou resetar o processador, a execução sempre começará a partir do endereço de memória inicial.

2.3. Unidade de Controle

A Unidade de Controle é o cérebro do processador como pode ser visto na imagem 6. Sua função é decodificar a instrução e gerar os sinais de controle necessários para governar as ações dos componentes do caminho de dados. Esse processo de decodificação geralmente ocorre em dois níveis para simplificar a lógica.

- **Decodificador Principal:** Analisa o `opcode` da instrução para determinar sua categoria (aritmética, acesso à memória, desvio, etc.). A partir disso, gera um conjunto de sinais de alto nível que habilitam ou desabilitam as principais partes do processador:
 - `RegWrite`: permite a escrita no banco de registradores
 - `MemRead / MemWrite`: controlam o acesso à memória de dados
 - `ALUSrc`: seleciona a segunda entrada da ULA (registrador ou valor imediato)
 - `Branch`: indica uma instrução de desvio
- **ALU Control:** Unidade de controle específica que refina a ação a ser executada. Combina um sinal geral vindo do decodificador principal (`ALUOp`) com os campos `func3` e `func7` da instrução para gerar o comando exato que a ULA deve realizar (soma, subtração ou operação lógica).

Essa abordagem em dois estágios torna o projeto mais modular e eficiente. Juntos, esses sinais orquestram o fluxo de dados através do processador, garantindo que cada instrução seja executada corretamente.

2.3.1. Controle Principal

A unidade de controle principal é um bloco combinacional que recebe como entrada principal o `opcode` da instrução e gera os sinais que determinam o comportamento dos demais componentes.

Como pode ser visto na figura 7 o módulo `control_unit` utiliza um bloco `always @(*)` para garantir que as saídas reajam instantaneamente a qualquer mudança

Implementing RISC-V Control Unit

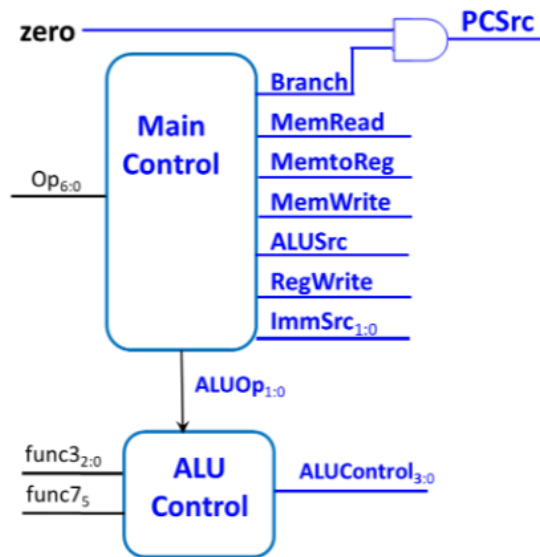


Figure 1: Single-cycle processor control unit

Table 1 Main Decoder truth table

Instruction	Opcode	ALUSrc	MemtoReg	RegWrite	MemRead	MemWrite	Branch	ALUOp
R-type	0110011	0	0	1	0	0	0	10
I-type ALU	0010011	1	0	1	0	0	0	10
lw	0000011	1	1	1	1	0	0	00
sw	0100011	1	X	0	0	1	0	00
beq	1100011	0	X	0	0	0	1	01

Figure 6. Unidade de Controle

```

1  `timescale 1ns / 1ps
2
3  // TODO: imm_src
4  module control_unit (
5      input wire [6:0] opcode,    // Instruction opcode
6
7      // EX stage control signals
8      output reg [1:0] ALUOp,    // ALU operation type
9      output reg ALUSrc,        // ALU source select (0=reg, 1=imm)
10
11     // MEM stage control signals
12     output reg Branch,        // Branch instruction
13     output reg Jump,          // Jump instruction (JAL/JALR)
14     output reg MemRead,       // Memory read enable
15     output reg MemWrite,      // Memory write enable
16
17     // WB stage control signals
18     output reg RegWrite,      // Register write enable
19     output reg MemtoReg       // Data source select (0=ALU, 1=Mem)
20 );

```

Figure 7. Código Fonte da Unidade de Controle Principal

no *opcode* de entrada. Dentro deste bloco, a lógica central é uma grande estrutura *case* (*opcode*), que funciona como o decodificador principal da instrução.

O processo de decodificação funciona da seguinte forma:

1. **Valores Padrão:** No início do bloco *always*, todos os sinais de controle são inicializados com um valor padrão, geralmente 0 (desativado). Isso garante que, por padrão, nenhuma operação de escrita em registrador, acesso à memória ou desvio ocorra. Os sinais só são ativados se o *opcode* corresponder a uma instrução que necessite daquela funcionalidade.
2. **Decodificação por opcode:** O *case* compara o *opcode* de entrada com os padrões definidos pela arquitetura RISC-V para cada tipo de instrução e ativa os sinais de controle apropriados:
 - **Instruções R-type (7'b0110011):** Ativa o sinal *RegWrite* para que o resultado da ULA seja salvo no banco de registradores. O sinal *ALUSrc* é mantido em 0 (padrão), fazendo com que a ULA use dois registradores como operandos. O *ALUOp* é configurado para 2'b10, indicando ao módulo *ALU Control* que a operação específica será definida pelos campos *funct3/funct7*.
 - **Instruções Load (7'b0000011):** Ativa *RegWrite* e *MemRead*. Além disso, aciona *ALUSrc* para que a ULA some o registrador base com o imediato a fim de calcular o endereço, e *MemtoReg* para que o dado lido da memória (e não o resultado da ULA) seja gravado no banco de registradores.
 - **Instruções Store (7'b0100011):** Ativa *ALUSrc* para o cálculo do endereço (similar ao Load), mas, em vez de ler, ativa *MemWrite* para escrever o dado do segundo registrador na memória. O *RegWrite* per-

manece desativado, pois a instrução de *store* não altera o banco de registradores.

- **Instruções Branch (7'b1100011):** Ativa o sinal *Branch*, que habilita a lógica de desvio condicional. O *ALUOp* é definido como 2'b01 para instruir a ULA a realizar uma subtração para a comparação, sem qualquer escrita em registrador ou memória.
- **Instruções de Salto (JAL/JALR):** Ativam o sinal *Jump* para indicar um desvio incondicional e *RegWrite* para salvar o endereço de retorno (PC+4) no registrador de destino.

Essa implementação mapeia diretamente o conjunto de instruções da arquitetura para os sinais de hardware necessários para orquestrar as operações no caminho de dados a cada ciclo de clock.

2.3.2. Controle da ULA

O controle da ULA (ALU Control) é um segundo bloco combinacional que gera o sinal final (ALU Control) para a ULA, utilizando o sinal *ALUOp* da unidade principal e os campos *funct3* e *funct7* da instrução.

```
1  `timescale 1ns / 1ps
2  `include "alu_defines.vh"
3
4  // ALUOp = 00: instruções de load/store → sempre ADD
5  // ALUOp = 01: instruções de branch → geralmente SUB, SLT, SLTU
6  // ALUOp = 10: R-type (funct7 + funct3 necessários)
7  // ALUOp = 11: I-type (funct3, e às vezes funct7)
8
9  module alu_control (
10     input wire [1:0] ALUOp,      // Sinal da control unit
11     input wire [9:0] Funct,      // {funct7[6:0], funct3[2:0]} da instrução
12     output reg [3:0] Op          // Operação para a ALU
13
14     // -----
15     // Conecte a esse modulo com:
16     // wire [9:0] Funct = {if_id_instr[31:25], if_id_instr[14:12]};
17     // -----
18 );
19
```

Figure 8. Código Fonte da Unidade de Controle da ULA

Analisando o código do módulo *alu_control*, vemos que ele recebe o sinal *ALUOp* (2 bits) da unidade de controle principal e um sinal *Funct* (10 bits), que combina o *funct7* e o *funct3* da instrução. A sua função é traduzir esses sinais de alto nível na operação específica de 4 bits que a ULA deve executar.

A lógica é implementada com uma estrutura de *case* aninhada dentro de um bloco combinacional *always @(*)*:

1. **Seleção por Categoria (case (ALUOp)):** O *case* externo seleciona o modo de operação com base no sinal *ALUOp* vindo do controle principal:

- $ALUOp = 2'b00$: Corresponde a instruções de *load*, *store* ou *AUIPC*. Nessas operações, a ULA é sempre usada para fazer uma soma (para calcular o endereço de memória ou somar ao PC). Portanto, a saída Op é diretamente definida como ALU_ADD .
- $ALUOp = 2'b01$: Usado para instruções de *branch*. Um case interno no campo $Func[2:0]$ (o $func3$) é usado para determinar a comparação a ser feita. Para *beq* e *bne* ($func3 = 3'b000$), a operação é ALU_SUB . Para *blt*/*bge* ($func3 = 3'b100, 3'b101$, etc.), as operações são ALU_SLT ou ALU_SLTU .
- $ALUOp = 2'b10$: Indica uma instrução *R-type*. Este é o caso mais complexo, onde a *ALU Control* precisa usar $func3$ e, às vezes, um bit de $func7$ para determinar a operação exata:
 - Para $func3 = 3'b000$, o bit $Func[9]$ (correspondente ao bit 30 da instrução) diferencia entre *ADD* (0) e *SUB* (1).
 - Para $func3 = 3'b101$, o mesmo bit diferencia entre deslocamento lógico *SRL* (0) e aritmético *SRA* (1).
 - Para outros valores, como $3'b111$ (*AND*), $3'b110$ (*OR*) e $3'b100$ (*XOR*), a operação é definida diretamente.
- $ALUOp = 2'b11$: Corresponde a instruções *I-type* (aritméticas com imediato). A lógica é semelhante à do *R-type*, mas aqui o case interno em $func3$ define operações como *ADDI*, *SLTI*, *XORI*, etc.

Em resumo, a *ALU Control* atua como um decodificador especializado. Ela recebe uma “dica” da unidade de controle principal ($ALUOp$) sobre qual família de instruções está sendo executada e, em seguida, analisa os campos de função da instrução para determinar e enviar o comando preciso que a ULA deve executar.

2.4. Tratamento de Hazards no Pipeline

Em uma arquitetura pipeline, podem ocorrer conflitos conhecidos como hazards (riscos), que impedem que a próxima instrução seja executada no ciclo de clock seguinte. O tratamento adequado desses hazards é fundamental para garantir a correção e o desempenho do processador. Esta seção descreve as unidades de hardware dedicadas a essa tarefa.

2.4.1. Unidade de Forwarding (Adiantamento)

A Unidade de Forwarding, ou Forwarding Unit, é responsável por resolver a maioria dos hazards de dados. Este tipo de hazard ocorre quando uma instrução precisa de um resultado que ainda não foi escrito de volta no banco de registradores por uma instrução anterior. Em vez de parar o pipeline (stall), a unidade de forwarding identifica essa dependência e “adianta” o resultado diretamente da saída dos estágios EX ou MEM para a entrada da ULA, evitando a espera.

A lógica compara os registradores de origem ($rs1, rs2$) da instrução no estágio ID com os registradores de destino (rd) das instruções nos estágios EX e MEM.

Analisando a imagem 9, podemos ver essa lógica em ação. O módulo é um bloco puramente combinacional que gera dois sinais de controle, *ForwardA* e *ForwardB*, que controlarão os multiplexadores na entrada da ULA.

```

1  `timescale 1ns / 1ps
2
3  module forwarding_unit (
4      input wire [4:0] EX_rs1,
5      input wire [4:0] EX_rs2,
6      input wire [4:0] MEM_rd,
7      input wire [4:0] WB_rd,
8      input wire      MEM_RegWrite,
9      input wire      WB_RegWrite,
10     output reg [1:0] ForwardA,
11     output reg [1:0] ForwardB
12 );
13
14     always @(*) begin
15         // Forward A (rs1)
16         if (MEM_RegWrite && (MEM_rd != 5'd0) && (MEM_rd == EX_rs1)) begin
17             ForwardA = 2'b10; // Forward from MEM
18         end else if (WB_RegWrite && (WB_rd != 5'd0) && (WB_rd == EX_rs1)) begin
19             ForwardA = 2'b01; // Forward from WB
20         end else begin
21             ForwardA = 2'b00; // No forwarding
22         end
23
24         // Forward B (rs2)
25         if (MEM_RegWrite && (MEM_rd != 5'd0) && (MEM_rd == EX_rs2)) begin
26             ForwardB = 2'b10; // Forward from MEM
27         end else if (WB_RegWrite && (WB_rd != 5'd0) && (WB_rd == EX_rs2)) begin
28             ForwardB = 2'b01; // Forward from WB
29         end else begin
30             ForwardB = 2'b00; // No forwarding
31         end
32     end
33
34 endmodule

```

Figure 9. Código Fonte da Unidade de Forward

A lógica de decisão está dividida em duas partes idênticas, uma para cada operando da ULA (`rs1` e `rs2`):

1. **Lógica para ForwardA (operando vindo de rs1):** O código implementa uma cadeia de prioridade para decidir de onde o dado deve vir:

- **Prioridade 1 – Adiantamento do Estágio MEM:** A primeira condição verifica se:

- A instrução no estágio MEM vai escrever em um registrador (`MEM_RegWrite == 1`);
- O registrador de destino não é o `x0` (que não pode ser modificado);
- O registrador de destino da instrução em MEM (`MEM_rd`) é o mesmo que o registrador de origem da instrução em EX (`EX_rs1`).

Se todas as condições forem verdadeiras, `ForwardA` é definido como `2'b10`, instruindo o multiplexador na entrada da ULA a selecionar o resultado vindo do estágio MEM.

- **Prioridade 2 – Adiantamento do Estágio WB:** Se a primeira condição for falsa, verifica-se uma dependência com o estágio de Escrita (WB). As condições são as mesmas, mas comparando `WB_RegWrite` e `WB_rd` com `EX_rs1`. Se houver dependência, `ForwardA` é definido como `2'b01`, sinalizando para o multiplexador usar o resultado do estágio WB.

- **Sem Adiantamento:** Se nenhuma das condições anteriores for atendida, significa que não há *hazard* de dados para este operando. Assim, `ForwardA` é definido como `2'b00`, e a ULA usará o valor lido normalmente do banco de registradores.

2. **Lógica para ForwardB (operando vindo de rs2):** As linhas correspondentes replicam a mesma lógica de prioridade, mas desta vez comparando `MEM_rd` e `WB_rd` com o segundo registrador de origem da instrução em EX (`EX_rs2`).

A ordem das verificações (`if / else if`) é crucial. A unidade sempre prioriza o adiantamento do estágio MEM sobre o WB, pois o resultado no estágio MEM é de uma instrução mais recente no pipeline, sendo, portanto, o valor mais atualizado disponível.

2.4.2. Unidade de Detecção de Hazards

A Unidade de Detecção de Hazards (Hazard Detection Unit) é responsável por identificar situações que não podem ser resolvidas apenas por forwarding e que exigem uma parada (stall) do pipeline. A principal situação é o hazard de load-use, que ocorre quando uma instrução tenta usar um dado que está sendo carregado da memória pela instrução imediatamente anterior (ex: `lw x5, 0(x2)` seguido por `add x6, x5, x7`).

Quando um hazard de load-use é detectado no estágio ID, esta unidade força uma bolha no pipeline. Isso é feito ao desabilitar a atualização do PC e do registrador de pipeline IF/ID por um ciclo, efetivamente "congelando" os estágios IF e ID enquanto uma instrução nop (bolha) é inserida no estágio EX.

O código-fonte da imagem 10 implementa essa lógica de forma concisa. Ele é um bloco puramente combinacional que gera um único sinal de saída, `stall`. A lógica para ativar este sinal, contida na linha correspondente do código, implementa exatamente as condições de um *hazard* do tipo *load-use*:

```

1  `timescale 1ns / 1ps
2  `include "constants.vh"
3
4  module hazard_detection (
5      // Sinais da instrução no estágio ID
6      input wire [4:0] if_id_rs1_addr, // Endereço do registrador fonte 1
7      input wire [4:0] if_id_rs2_addr, // Endereço do registrador fonte 2
8
9      // Sinais da instrução no estágio EX (vindos do registrador ID/EX)
10     input wire [4:0] id_ex_rd_addr, // Endereço do registrador destino
11     input wire id_ex_mem_read,      // Sinal CTRL_MEM_READ do estágio EX
12
13     // Saída de controle
14     output reg stall                // Sinal de stall para o pipeline
15 );
16
17 always @(*) begin
18     // Hazard de load-use ocorre quando:
19     // 1. Instrução em EX é um load (mem_read=1)
20     // 2. O registrador destino não é x0
21     // 3. O registrador destino é um dos operandos da instrução atual
22     stall = id_ex_mem_read &&
23             (id_ex_rd_addr != 5'b0) &&
24             ((id_ex_rd_addr == if_id_rs1_addr) || (id_ex_rd_addr == if_id_rs2_addr));
25 end
26
27 endmodule

```

Figure 10. Código Fonte do Controle de Hazard

- **A instrução é um *load* no estágio EX:** Isso é verificado pelo sinal `id_ex_mem_read`. Se este sinal estiver ativo, significa que a instrução que acabou de passar pelo estágio ID e agora está no EX é uma instrução de leitura da memória (por exemplo, `lw`).
- **O destino do *load* é um dos operandos da instrução atual:** A expressão `(id_ex_rd_addr == if_id_rs1_addr) || (id_ex_rd_addr == if_id_rs2_addr)` verifica se o registrador de destino da instrução de *load* (`id_ex_rd_addr`) é o mesmo que um dos registradores fonte (`if_id_rs1_addr` ou `if_id_rs2_addr`) da instrução que está logo atrás, no estágio ID.

Essencialmente, a unidade está verificando: “A instrução no estágio EX está lendo da memória para um registrador que a instrução no estágio ID precisa usar agora?”

Se a resposta for afirmativa, o sinal `stall` é ativado. Este sinal é então usado pela lógica de controle do pipeline para parar os estágios IF e ID por um ciclo, permitindo que a instrução de *load* avance para o estágio MEM. Após esse ciclo de espera, o dado carregado da memória estará pronto para ser adiantado (*forwarded*) para a instrução seguinte, que foi mantida no estágio ID e agora pode prosseguir sem problemas.

2.4.3. Unidade de Desvio (Branch Unit)

A Unidade de Desvio é um componente fundamental no estágio de Execução, responsável por implementar a lógica dos desvios condicionais, como `beq` (branch on equal) e `bne` (branch on not equal). Sua principal função é determinar se um desvio deve ou não ser

tomado e, em caso afirmativo, calcular o endereço de destino desse desvio.

```
1  `timescale 1ns / 1ps
2  `include "constants.vh"
3
4  module branch_unit (
5      input wire [31:0] id_ex_pc,
6      input wire [31:0] id_ex_instruction,
7      input wire [31:0] id_ex_rs1_data,
8      input wire [31:0] id_ex_rs2_data,
9      input wire [31:0] id_ex_immediate,
10     input wire [CONTROL_SIGNALS_WIDTH-1:0] id_ex_control_signals,
11
12     output reg pc_src,
13     output reg [31:0] new_pc,
14     output reg flush
15 );
16
17 // Extrair sinais de controle e campos da instrução
18 wire is_branch = id_ex_control_signals[CTRL_BRANCH]; // Usando macro com prefixo CTRL_
19 wire [2:0] funct3 = id_ex_instruction[14:12]; // Campo funct3 para tipo de branch
20 wire is_jump = id_ex_control_signals[CTRL_JUMP]; // Sinal de instrução jump
21
22 always @(*) begin
23     // Valores padrão
24     pc_src = 1'b0;
25     new_pc = 32'b0;
26     flush = 1'b0;
27
28     // Lógica para branches
29     if (is_branch) begin
30         case (funct3)
31             `BRANCH_EQ: pc_src = (id_ex_rs1_data == id_ex_rs2_data); // beq
32             `BRANCH_NE: pc_src = (id_ex_rs1_data != id_ex_rs2_data); // bne
33             `BRANCH_LT: pc_src = ($signed(id_ex_rs1_data) < $signed(id_ex_rs2_data)); // blt
34             `BRANCH_GE: pc_src = ($signed(id_ex_rs1_data) >= $signed(id_ex_rs2_data)); // bge
35             `BRANCH_LTU: pc_src = (id_ex_rs1_data < id_ex_rs2_data); // bltu
36             `BRANCH_GEU: pc_src = (id_ex_rs1_data >= id_ex_rs2_data); // bgeu
37             default: pc_src = 1'b0;
38         endcase
39
40         if (pc_src) begin
41             new_pc = id_ex_pc + id_ex_immediate;
42             flush = 1'b1;
43         end
44     end
45
46     // Lógica para jumps (JAL/JALR)
47     else if (is_jump) begin
48         pc_src = 1'b1;
49         new_pc = id_ex_pc + id_ex_immediate; // Para JALR, o cálculo é diferente
50         flush = 1'b1;
51     end
52 end
53 endmodule
```

Figure 11. Código fonte da Branch Unit

Analisando o código da imagem 11 o módulo `branch_unit` recebe diversas informações do estágio anterior (ID/EX), tais como:

- PC da instrução: `id_ex_pc`
- Dados dos registradores a serem comparados: `id_ex_rs1_data` e `id_ex_rs2_data`
- Valor imediato para o cálculo do desvio: `id_ex_immediate`
- Sinais de controle: `id_ex_control_signals`

O comportamento da unidade é implementado em um bloco combinacional `always @(*)`, que realiza as seguintes ações:

- **Avaliação da Condição de Desvio:** A lógica principal para os desvios condicionais reside em um `case` que avalia o campo `funct3` da instrução. Para cada tipo de desvio, é realizada uma comparação entre `id_ex_rs1_data` e `id_ex_rs2_data`:

- **BRANCH_EQ (beq):** Verifica a igualdade (==).
- **BRANCH_NE (bne):** Verifica a desigualdade (!=).
- **BRANCH_LT / BRANCH_LTU (blt/bltu):** Compara se o primeiro operando é menor que o segundo, considerando valores com sinal (\$signed) e sem sinal.
- **BRANCH_GE / BRANCH_GEU (bge/bgeu):** Verifica se o primeiro operando é maior ou igual ao segundo, tanto com sinal quanto sem sinal.

Se a condição for verdadeira, o sinal de saída `pc_src` é ativado (1'b1), indicando à próxima etapa do processador que o desvio deve ser tomado.

- **Cálculo do Endereço de Destino:** Caso um desvio seja tomado (`pc_src = 1'b1`) ou se a instrução for um salto (`is_jump`), a unidade calcula o endereço de destino somando o PC da instrução atual (`id_ex_pc`) com o valor do imediato (`id_ex_immediate`). O resultado é colocado na saída `new_pc`.
- **Sinalização de Bolha (Flush):** Quando um desvio é tomado ou um salto ocorre, as instruções que já entraram no pipeline sequencialmente (como a instrução em `PC+4`) estão incorretas e precisam ser descartadas. O módulo `branch_unit` gera o sinal `flush` para indicar que essas instruções devem ser invalidadas (transformadas em "bolhas").

2.5. Sistema de Memória

O processador interage com dois blocos de memória principais: a memória de instruções, de onde as instruções são buscadas, e a memória de dados, usada para operações de carga (*load*) e armazenamento (*store*).

2.5.1. Memória de Instruções

A memória de instruções é uma memória somente de leitura (do ponto de vista do processador) que armazena o programa a ser executado. Ela recebe um endereço de 32 bits do Contador de Programa (PC) e retorna a instrução de 32 bits correspondente de forma assíncrona.

A partir do código da Figura 12, podemos ver exatamente como essas características são implementadas:

1. **Estrutura Parametrizada:** O módulo é definido com parâmetros que o tornam flexível: `WIDTH` confirma a largura de 32 bits para cada instrução, `DEPTH` define a capacidade total de armazenamento (neste caso, 256 instruções), e `INIT_FILE` permite especificar um arquivo de inicialização para pré-carregar o programa.
2. **Leitura Assíncrona (Combinacional):** A operação de leitura é implementada com a instrução `assign`, criando um circuito puramente combinacional. Isso significa que não há necessidade de um sinal de clock para a leitura: a saída `instr` é atualizada instantaneamente sempre que a entrada `instr_addr` muda, garantindo a velocidade necessária para o estágio de busca (*fetch*) do processador.
3. **Conversão de Endereço:** A lógica de endereçamento `memory[instr_addr[31:2]]` é um detalhe crucial. O PC fornece um endereço de byte, mas como cada instrução RISC-V ocupa 4 bytes (32 bits), a memória interna é organizada por palavras. A expressão `instr_addr[31:2]`

```

1  `timescale 1ns / 1ps
2
3  module instruction_memory #(
4      parameter WIDTH      = 32,          // Largura de cada instrução
5      parameter DEPTH      = 256,        // Quantidade de instruções
6      parameter INIT_FILE  = ""          // Caminho do arquivo de inicialização (HEX)
7  )(
8      input wire [31:0] instr_addr,      // Endereço da instrução (PC)
9
10     output wire [WIDTH-1:0] instr      // Instrução buscada
11 );
12
13     // Memória de instruções
14     reg [WIDTH-1:0] memory [0:DEPTH-1];
15
16     // Leitura combinacional
17     assign instr = memory[instr_addr[31:2]]; // addr >> 2
18
19     // Inicialização opcional com arquivo
20     initial begin
21         if (INIT_FILE != "") begin
22             $display("Loading instruction memory from %s", INIT_FILE);
23             $readmemh(INIT_FILE, memory);
24         end
25     end
26
27 endmodule

```

Figure 12. Código Fonte da Memória de Instruções

seleciona os bits 31 a 2 do endereço, equivalente a uma divisão por 4, convertendo o endereço de byte do PC para o índice de palavra correto.

4. **Inicialização em Simulação:** O bloco `initial` é usado apenas em simulação. Ele verifica se um caminho de arquivo foi fornecido em `INIT_FILE` e, caso afirmativo, utiliza a função de sistema `$readmemh` para ler um arquivo de texto contendo o programa em formato hexadecimal, carregando-o na memória antes do início da simulação. É assim que o “software” é inserido no “hardware” para teste e verificação.

2.5.2. Memória de Dados

A memória de dados armazena as variáveis e estruturas de dados utilizadas pelo programa. Ela suporta tanto operações de leitura quanto de escrita, controladas pelos sinais `MemRead` e `MemWrite`. A escrita é síncrona, ocorrendo na borda de subida do clock para evitar condições de corrida.

O código-fonte da imagem 13 implementa essa funcionalidade de forma clara, separando a lógica de leitura e escrita para atender aos requisitos de um pipeline de processador:

1. **Portas de Interface:** O módulo possui portas para o endereço (`mem_addr`), o dado a ser escrito (`write_data`) e o dado lido (`read_data`). Crucialmente, ele recebe os sinais de controle `mem_read` e `mem_write` da Unidade de Controle, além de um sinal de `clk` para sincronização.

```

1  `timescale 1ns / 1ps
2
3  module data_memory #(
4      parameter WIDTH = 32,           // Largura de cada palavra (em bits)
5      parameter DEPTH = 1024,        // Número de posições de memória
6      parameter INIT_FILE = "none"   // Arquivo de inicialização (opcional)
7  ) (
8      input wire clk,                 // Sinal de clock
9
10     input wire [WIDTH-1:0] mem_addr, // Endereço de memória (vindo da ALU)
11     input wire [WIDTH-1:0] write_data, // Dado para escrita
12     input wire mem_write,            // Sinal de escrita
13     input wire mem_read,             // Sinal de leitura
14     output reg [WIDTH-1:0] read_data // Dado lido
15 );
16
17     localparam ADDR_WIDTH = $clog2(DEPTH);
18     reg [WIDTH-1:0] memory [0:DEPTH-1];
19
20     // Inicialização
21     integer i;
22     initial begin
23         // Inicializa toda a memória com zeros
24         for (i = 0; i < DEPTH; i = i + 1) begin
25             memory[i] = 32'h0;
26         end
27
28         if (INIT_FILE != "none") begin
29             $readmemh(INIT_FILE, memory);
30         end
31     end
32
33     // CORREÇÃO: Lógica separada para leitura e escrita
34     wire [ADDR_WIDTH-1:0] word_addr = mem_addr[$clog2(DEPTH)+1:2]; // Endereço de palavra
35
36     // Leitura combinacional (para compatibilidade com pipeline)
37     always @(*) begin
38         if (mem_read) begin
39             read_data = memory[word_addr];
40         end else begin
41             read_data = 32'h0;
42         end
43     end

```

Figure 13. Código Fonte da Memória de Dados

2. **Escrita Síncrona:** A operação de escrita é implementada dentro de um bloco `always @(posedge clk)`, tornando-a síncrona. A escrita na memória (`memory[word_addr] <= write_data;`) só acontece se:

- Ocorre uma borda de subida no clock;
- O sinal de controle `mem_write` está ativo (1).

Essa abordagem garante que a escrita na memória ocorra em um ponto bem definido do ciclo de clock, evitando instabilidades e condições de corrida com outras operações, como a leitura.

3. **Leitura Combinacional:** A lógica de leitura é implementada em um bloco `always @(*)`, caracterizando-a como combinacional (assíncrona).

- Se o sinal `mem_read` estiver ativo (1), a saída `read_data` recebe o valor da posição de memória endereçada por `word_addr`;
- Caso contrário, a saída é mantida em zero.

A leitura combinacional permite que o dado esteja disponível no mesmo ciclo em que é solicitado, compatível com o estágio de Acesso à Memória (MEM) do pipeline.

4. **Endereçamento de Palavra:** Assim como na memória de instruções, o endereço de byte (`mem_addr`) vindo da ULA é convertido para um endereço de palavra (`word_addr`), selecionando os bits superiores para indexar corretamente o vetor de memória.

2.6. Implementação dos Estágios do Pipeline

A implementação do pipeline é realizada através de uma série de registradores intermediários que separam os cinco estágios. Cada um desses módulos sequenciais é responsável por capturar e armazenar os resultados de um estágio ao final de um ciclo de clock, disponibilizando-os como entrada para o estágio seguinte no próximo ciclo. Essa estrutura garante a sincronização e o fluxo correto de dados e sinais de controle através do processador.

2.6.1. Registrador IF/ID

Ao final do estágio de Busca, a instrução lida da memória e o endereço da próxima instrução (`PC+4`) são capturados pelo registrador IF/ID. Este módulo garante que esses valores sejam passados de forma estável para o estágio de Decodificação no próximo ciclo de clock.

O código Verilog da imagem 14 mostra que o registrador IF/ID, além de simplesmente armazenar dados, ele desempenha um papel ativo no controle do pipeline, respondendo aos sinais `rst`, `flush` e `stall`. A lógica síncrona implementada no bloco `always` define seu comportamento de acordo com diferentes situações do ciclo de clock.

Durante o **reset** (`if (rst)`), o registrador é limpo, forçando as saídas `pc_out` e `instr_out` a zero. Esse procedimento garante que, na inicialização do processador, o estado seja conhecido e válido, evitando a propagação de instruções indesejadas para os estágios seguintes.

No caso de um **flush** (`else if (flush)`), geralmente causado por uma instrução de desvio tomada em um estágio posterior, a instrução buscada logo após

```

1  `timescale 1ns / 1ps
2
3  module if_id (
4      input wire      clk,
5      input wire      rst,
6
7      // Controle
8      input wire      stall,
9      input wire      flush,
10
11     // Entradas do estágio IF
12     input wire [31:0] pc_in,      // PC atual vindo do IF
13     input wire [31:0] instr_in,  // Instrução da memória de instruções
14
15     // Saídas para o estágio ID
16     output reg [31:0] pc_out,     // PC passado para o ID/EX
17     output reg [31:0] instr_out  // Instrução passada para os módulos de ID
18 );
19
20 always @(posedge clk or posedge rst) begin
21     if (rst) begin
22         pc_out    <= 32'b0;
23         instr_out <= 32'b0;
24     end else if (flush) begin
25         pc_out    <= 32'b0;
26         instr_out <= 32'b0;
27     end else if (!stall) begin
28         pc_out    <= pc_in;
29         instr_out <= instr_in;
30     end
31     // Quando stall está ativo, mantemos os valores anteriores
32 end
33
34 endmodule

```

Figure 14. Código Fonte do Registrador IF/ID

o desvio — conhecida como *branch delay slot instruction* — precisa ser descartada. O sinal `flush` aciona a lógica que zera o conteúdo do registrador, inserindo o valor `0x00000000` (normalmente associado a uma `nop` ou instrução inválida) no estágio ID. Dessa forma, a instrução incorreta é efetivamente anulada antes de ser decodificada.

Em condições normais de operação (`else if (!stall)`), quando não há atrasos no pipeline, o registrador captura os valores de entrada `pc_in` e `instr_in`, armazenando-os em `pc_out` e `instr_out`, respectivamente. Assim, a instrução buscada no estágio IF avança para o estágio ID de forma contínua.

O **stall**, por sua vez, é tratado de forma implícita. Quando o sinal está ativo, nenhuma das condições anteriores é satisfeita e, conseqüentemente, o registrador mantém seus valores atuais. Isso impede que uma nova instrução (potencialmente causadora de hazard) entre no estágio ID e, ao mesmo tempo, permite que o Contador de Programa (PC) seja congelado. Como resultado, a mesma instrução será buscada novamente no próximo ciclo, garantindo a correta sincronização e resolução do hazard antes da retomada do fluxo normal do pipeline.

2.6.2. Registrador ID/EX

Após a decodificação da instrução e a leitura dos operandos, o registrador ID/EX armazena uma grande quantidade de informações para o estágio de Execução. Isso inclui todos os sinais de controle, os valores lidos do banco de registradores, o imediato estendido e os índices dos registradores (`rs1`, `rs2`, `rd`).

O código da imagem 15 revela que este componente vai além de um simples armazenamento: ele é um elemento ativo no controle do fluxo do pipeline, respondendo a sinais de `stall` e `flush`. Sua operação síncrona, definida por um bloco `always @(posedge clk or posedge rst)`, apresenta comportamentos distintos conforme as condições de controle.

Na **operação normal**, quando o sinal `stall` está inativo (`!stall`), o registrador cumpre sua função principal. Na borda de subida do clock, ele captura todos os dados e sinais de controle provenientes do estágio de Decodificação (prefixo `id_`) e os propaga para suas saídas, que alimentarão o estágio de Execução (prefixo `ex_`). Entre esses dados estão os valores lidos (`id_read_data_1`, `id_read_data_2`), o valor do PC (`id_pc_out`), o imediato (`id_imm_data`) e todos os sinais gerados pela Unidade de Controle.

Quando ocorre **reset ou flush** (`if (rst || flush)`), seja por inicialização ou por uma instrução de desvio tomada no estágio EX, o registrador anula a instrução recém-decodificada, forçando todas as suas saídas a zero. Mais importante, todos os sinais de controle (`ex_RegWrite`, `ex_MemWrite`, etc.) são desativados. Esse procedimento insere uma bolha (`nop`) no estágio de Execução, garantindo que a instrução descartada não produza efeitos indevidos.

No caso de um **stall**, quando a Unidade de Detecção de Hazards ativa o sinal `stall`, a condição `if (!stall)` não é satisfeita. Como não há um bloco `else`, o registrador mantém seus valores atuais, ignorando as novas entradas provenientes do estágio ID. Dessa forma, a instrução permanece no estágio de Decodificação até que o

```

1  `timescale 1ns / 1ps
2
3  module id_ex (
4      input wire clk,
5      input wire rst,
6      input wire stall,
7      input wire flush,
8
9      // Entradas do estágio ID
10     input wire [31:0] id_read_data_1, // Dado lido do registrador rs1
11     input wire [31:0] id_read_data_2, // Dado lido do registrador rs2
12     input wire [31:0] id_imm_data,    // Valor imediato estendido
13     input wire [31:0] id_pc_out,      // PC atual (antes do +4)
14     input wire [4:0] id_rs1,          // Endereço do registrador rs1
15     input wire [4:0] id_rs2,          // Endereço do registrador rs2
16     input wire [4:0] id_rd,           // Endereço do registrador rd
17     input wire [2:0] id_funct3,       // Função 3 da instrução
18     input wire [6:0] id_funct7,       // Função 7 da instrução
19
20     // Sinais de controle da control_unit
21     input wire id_ALUSrc,             // Controle do mux da ULA
22     input wire [1:0] id_ALUOp,        // Controle da operação da ULA
23     input wire id_Branch,             // Sinal de branch
24     input wire id_Jump,               // Sinal de jump (JAL/JALR)
25     input wire id_MemRead,            // Sinal de leitura de memória
26     input wire id_MemWrite,           // Sinal de escrita em memória
27     input wire id_RegWrite,           // Sinal de escrita no banco de registradores
28     input wire id_MemtoReg,           // Controle do mux WB (memória ou ULA para registrador)
29
30     // Saídas para o estágio EX
31     output reg [31:0] ex_read_data_1, // Para entrada A da ULA
32     output reg [31:0] ex_read_data_2, // Para mux da entrada B da ULA
33     output reg [31:0] ex_imm_data,     // Para mux da ULA e shift left
34     output reg [31:0] ex_pc_out,       // PC para cálculo de branch
35     output reg [4:0] ex_rs1,           // Endereço rs1 (para forwarding)
36     output reg [4:0] ex_rs2,           // Endereço rs2 (para forwarding)
37     output reg [4:0] ex_rd,            // Endereço rd (para WB)
38     output reg [2:0] ex_funct3,        // Função 3 da instrução
39     output reg [6:0] ex_funct7,        // Função 7 da instrução
40
41     // Sinais de controle para os estágios EX/MEM/WB
42     output reg ex_ALUSrc,              // Sinal de controle ALUSrc
43     output reg [1:0] ex_ALUOp,         // Sinal de controle ALUOp
44     output reg ex_Branch,              // Para estágio MEM (controle de branch)
45     output reg ex_Jump,                // Para estágio MEM (controle de jump)
46     output reg ex_MemRead,             // Para estágio MEM (leitura de memória)
47     output reg ex_MemWrite,            // Para estágio MEM (escrita em memória)
48     output reg ex_RegWrite,            // Para estágio WB (escrita em registrador)
49     output reg ex_MemtoReg,            // Para estágio WB (seleção de dado para registrador)
50 );

```

Figure 15. Código Fonte do Registrador ID/EX

dado necessário esteja disponível para ser adiantado (*forwarded*), o que é fundamental para resolver o hazard de *load-use*.

2.6.3. Registrador EX/MEM

O registrador EX/MEM serve como ponte para o estágio de Memória. Ele armazena o resultado calculado pela ULA (seja um valor aritmético ou um endereço de memória), o dado a ser escrito na memória (no caso de uma instrução *store*), e os sinais de controle relevantes para os estágios MEM e WB.

```
1  `timescale 1ns / 1ps
2
3  module ex_mem (
4      input wire      clk,
5      input wire      rst,
6      input wire      stall,
7      input wire      flush,
8
9      // Control signals from EX stage (to be passed to MEM/WB stages)
10     // MEM stage control signals
11     input wire      ex_Branch,          // To MEM stage for branch decision
12     input wire      ex_Jump,            // To MEM stage for branch decision
13     input wire      ex_MemRead,         // To MEM stage for memory read
14     input wire      ex_MemWrite,        // To MEM stage for memory write
15     // WB stage control signals
16     input wire      ex_RegWrite,        // To WB stage for register write
17     input wire      ex_MemtoReg,        // To WB stage for memory-to-register mux
18
19     // Entradas do estágio EX
20     input wire [31:0] ex_adder_out,      // PC + (imm << 1) (vem do somador de branch)
21     input wire [31:0] ex_result,         // Resultado da ALU (vai para memória de dados)
22     input wire      ex_alu_zero,         // Indica se o segundo operando da ALU é imediato
23     input wire [4:0] ex_rd,              // Registrador destino (vem de id_ex)
24     input wire [31:0] ex_read_data_2_mux, // Dado para escrita (vem do mux de forwarding)
25
26     // Saídas para o estágio MEM
27     output reg [31:0] mem_addr,           // Endereço para memória de dados (ALU Result)
28     output reg      mem_alu_zero,         // Indica se o segundo operando da ALU é imediato
29     output reg [31:0] mem_write_data,     // Dado para escrita na memória
30     output reg [4:0] mem_rd,              // Registrador destino (para WB e forwarding)
31     output reg [31:0] mem_adder_out,      // PC + offset (para cálculo de branch)
32
33     // Control signals to MEM stage
34     output reg      mem_Branch,          // Branch control to MEM stage
35     output reg      mem_Jump,            // Branch control to MEM stage
36     output reg      mem_MemRead,         // Memory read control to MEM stage
37     output reg      mem_MemWrite,        // Memory write control to MEM stage
38
39     // Control signals to WB stage (passed through MEM stage)
40     output reg      mem_RegWrite,        // Register write control to WB stage
41     output reg      mem_MemtoReg,        // Memory-to-register mux control to WB stage
42  );
```

Figure 16. Código Fonte do Registrador EX/MEM

Analisando o código da imagem 16, observa-se que ele é um registrador síncrono que atualiza seus valores na borda de subida do clock (`posedge clk`) e incorpora uma lógica de controle capaz de lidar com diferentes situações do pipeline.

Na **operação normal**, quando não há sinais de controle como `stall` ou `flush`, o registrador cumpre sua função primária: capturar os resultados do estágio de Execução. Assim, na borda de subida do clock, os valores das entradas (`ex_result`,

`ex_read_data_2_mux`, `ex_rd`, etc.) são transferidos para as saídas correspondentes (`mem_addr`, `mem_write_data`, `mem_rd`, etc.). Esse mecanismo garante que os dados e os sinais de controle da instrução avancem de forma sincronizada para o estágio MEM.

Quando ocorre **reset ou flush**, a lógica implementada na linha 45 (`if (rst || flush)`) assume prioridade máxima. Caso um reset seja acionado ou a unidade de controle sinalize um flush — normalmente em consequência de um desvio ou salto tomado —, o registrador anula a instrução em processamento. Isso é feito zerando todos os campos de dados e desativando todos os sinais de controle (`mem_Branch`, `mem_MemWrite`, `mem_RegWrite`, etc.). Essa operação insere uma “bolha” (`nop`) no pipeline, prevenindo que a instrução invalidada produza qualquer efeito nos estágios seguintes.

No caso de um **stall**, o registrador só atualiza seus valores se o sinal `stall` estiver inativo (`if (!stall)`). Se a Unidade de Detecção de Hazards ativar o `stall`, essa condição será falsa e, como não há um bloco `else`, o registrador mantém seus valores atuais. Esse congelamento preserva o conteúdo do estágio EX/MEM por um ciclo, permitindo que a instrução anterior (causadora do hazard) avance, enquanto a instrução atual permanece no estágio EX, resolvendo a dependência de dados.

2.6.4. Registrador MEM/WB

Este é o último registrador do pipeline, responsável por levar os dados para o estágio final de Escrita. Ele armazena o resultado final da operação — que pode ser o valor vindo da ULA ou o dado lido da memória de dados — e o endereço do registrador de destino onde este valor será escrito.

O código da imagem 17 mostra um registrador síncrono que opera de forma muito similar aos seus predecessores no pipeline, respondendo aos mesmos sinais de controle para garantir a integridade do fluxo de instruções.

Sua principal função é transportar para o estágio WB dois dados essenciais: o resultado da ULA (`mem_result`) e o dado lido da memória (`mem_read_data`). A escolha de qual dos dois será utilizado não é feita neste estágio, mas sim no WB, por meio de um multiplexador controlado pelo sinal `MemtoReg`, que também é propagado por este registrador. Além disso, são transportados o endereço do registrador de destino (`mem_rd`) e o sinal de habilitação de escrita `mem_RegWrite`.

Na **operação normal**, quando o sinal `stall` está inativo (`!stall`), o registrador captura, na borda de subida do clock, os valores de suas entradas (prefixo `mem_`) e os disponibiliza em suas saídas (prefixo `wb_`). Isso permite que a instrução avance do estágio de Acesso à Memória para o estágio de Escrita de forma sincronizada.

No caso de **reset ou flush** (`if (rst || flush)`), todas as saídas do registrador são zeradas. A ação mais crítica é a desativação do sinal `wb_RegWrite`, funcionando como uma última barreira de segurança para impedir que uma instrução anulada por um desvio possa, sob qualquer circunstância, realizar uma escrita indevida no banco de registradores no estágio final.

Quando ocorre um **stall**, o registrador MEM/WB mantém seus valores atuais, congelando a instrução neste ponto do pipeline. Embora a maioria dos *stalls* seja resolvida em

```

1  `timescale 1ns / 1ps
2
3  module mem_wb (
4      input wire      clk,
5      input wire      rst,
6      input wire      stall,
7      input wire      flush,
8
9      // Entradas do estágio MEM
10     input wire [31:0] mem_read_data,    // Dado lido da memória (vem do data_memory)
11     input wire [31:0] mem_result,       // Resultado da ALU (vem do ex_mem)
12     input wire [4:0]  mem_rd,           // Registrador destino (vem do ex_mem)
13     input wire        mem_RegWrite,     // Sinal de escrita no banco de registradores
14     input wire        mem_MemtoReg,     // Controle do mux WB
15
16     // Saídas para o estágio WB
17     output reg [31:0] wb_read_data,     // Dado lido da memória (para o mux WB)
18     output reg [31:0] wb_result,        // Resultado da ALU (para o mux WB)
19     output reg [4:0]  wb_rd,            // Registrador destino
20     output reg        wb_RegWrite,      // Sinal de escrita no banco de registradores
21     output reg        wb_MemtoReg,      // Controle do mux WB
22 );
23
24 always @(posedge clk or posedge rst) begin
25     if (rst || flush) begin
26         // Reset/flush: zera todos os registradores
27         wb_read_data <= 32'b0;
28         wb_result    <= 32'b0;
29         wb_rd        <= 5'b0;
30         wb_RegWrite  <= 1'b0;
31         wb_MemtoReg  <= 1'b0;
32     end
33     else if (!stall) begin
34         // Registra os valores normalmente
35         wb_read_data <= mem_read_data;
36         wb_result    <= mem_result;
37         wb_rd        <= mem_rd;
38         wb_RegWrite  <= mem_RegWrite;
39         wb_MemtoReg  <= mem_MemtoReg;
40     end
41     // Se stall, mantém os valores atuais
42 end
43
44 endmodule

```

Figure 17. Código Fonte do Registrador MEM/WB

estágios anteriores, essa capacidade garante consistência no comportamento do pipeline como um todo.

Ao final de cada ciclo de clock, os valores estabilizados nas saídas deste registrador (`wb_result`, `wb_read_data`, `wb_rdd`, `wb_RegWrite`, `wb_MemtoReg`) contêm todas as informações necessárias para que o estágio de Write Back finalize a execução da instrução, seja atualizando um registrador ou não realizando nenhuma ação.

3. Resultados

Durante o desenvolvimento da pipeline para o processador RISC-V32I, como exibido pela forma de onda em 18 e testbranch em 19, 20 e 21, identificamos diversos problemas que afetam o funcionamento correto do processador em determinadas situações.

3.1. Funcionamento Parcial das Instruções

Instruções do tipo ADDI e LW funcionam corretamente na maioria dos casos, demonstrando que o caminho de dados básico está operacional. No entanto, mesmo com a presença da *Forwarding Unit*, observamos que, em alguns cenários, ainda é necessário inserir instruções NOP manualmente para evitar *hazards*. Isso indica que a unidade de *forwarding* não está cobrindo todos os casos de dependências de dados, possivelmente devido a lacunas na detecção ou no roteamento dos dados entre estágios.

3.2. Problemas de Controle em Branches

O maior problema encontrado está relacionado às instruções de desvio (BEQ, BNE, etc.). Atualmente, o PC não salta para o endereço correto quando ocorre um branch, e a pipeline não realiza o *flush* dos estágios posteriores de forma adequada. Isso resulta em execução incorreta de instruções que deveriam ser descartadas, quebrando o fluxo esperado do programa.

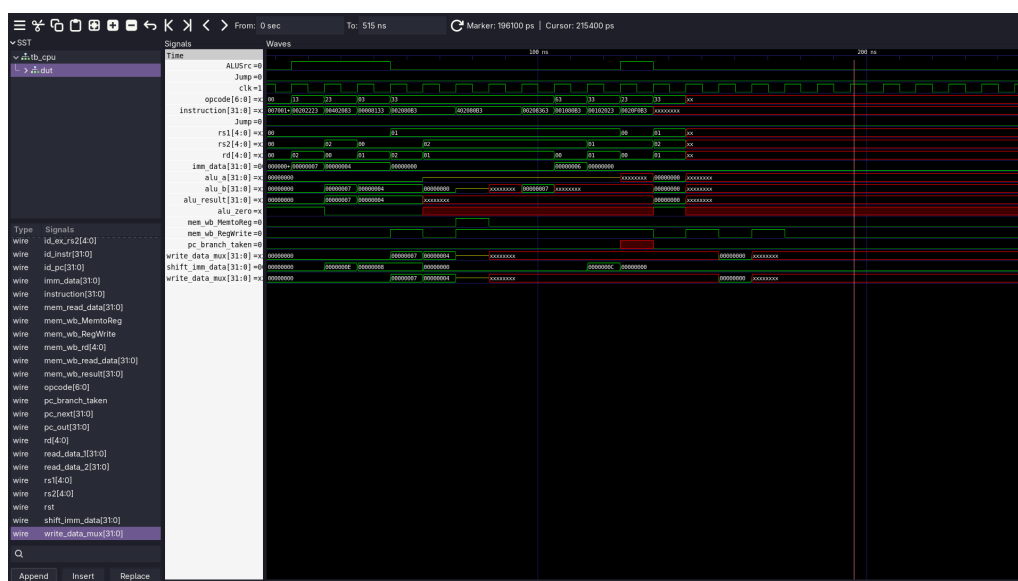


Figure 18. WAVE

```
=====
MEMÓRIA DE DADOS - Primeiras 16 posições:
=====
Addr | Hex Value | Dec Value
-----|-----|-----
00000000 | 0xZZZZZZZZ | Z
00000004 | 0xZZZZZZZZ | Z
00000008 | 0x00000000 | 0
0000000c | 0x00000000 | 0
00000010 | 0x00000000 | 0
00000014 | 0x00000000 | 0
00000018 | 0x00000000 | 0
0000001c | 0x00000000 | 0
00000020 | 0x00000000 | 0
00000024 | 0x00000000 | 0
00000028 | 0x00000000 | 0
0000002c | 0x00000000 | 0
00000030 | 0x00000000 | 0
00000034 | 0x00000000 | 0
00000038 | 0x00000000 | 0
0000003c | 0x00000000 | 0
=====
SIMULAÇÃO COMPLETA!
=====
```

Figure 19. Memória de Dados

MEMÓRIA DE INSTRUÇÕES - Primeiras 16 posições:		
Addr	Instrução (Hex)	Instrução (Bin)
00000000	00700113	000000000110000000000100010011
00000004	00202223	000000000100000001000100010011
00000008	00402083	000000001000000010000010000011
0000000c	00008133	00000000000000001000000100110011
00000010	002080b3	00000000001000001000000010110011
00000014	002080b3	00000000001000001000000010110011
00000018	402080b3	0100000001000001000000010110011
0000001c	402080b3	0100000001000001000000010110011
00000020	00208363	00000000001000001000001101100011
00000024	001080b3	0000000000100001000000010110011
00000028	00102023	000000000010000001000000100011
0000002c	0020f0b3	00000000001000001111000010110011
00000030	0000e0b3	00000000000000001110000010110011
00000034	00102023	00000000000100000010000000100011
00000038	xxxxxxxx	xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
0000003c	xxxxxxxx	xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx

Figure 20. Memória de Instruções

ESTADO FINAL DOS REGISTRADORES:		
Reg	Hex Value	Dec Value
x0	0xxxxxxxxx	x
x1	0xxxxxxxxx	x
x2	0xxxxxxxxx	x
x3	0x00000000	0
x4	0x00000000	0
x5	0x00000000	0
x6	0x00000000	0
x7	0x00000000	0
x8	0x00000000	0
x9	0x00000000	0
x10	0x00000000	0
x11	0x00000000	0
x12	0x00000000	0
x13	0x00000000	0
x14	0x00000000	0
x15	0x00000000	0
x16	0x00000000	0
x17	0x00000000	0
x18	0x00000000	0
x19	0x00000000	0
x20	0x00000000	0
x21	0x00000000	0
x22	0x00000000	0
x23	0x00000000	0
x24	0x00000000	0
x25	0x00000000	0
x26	0x00000000	0
x27	0x00000000	0
x28	0x00000000	0
x29	0x00000000	0
x30	0x00000000	0
x31	0x00000000	0

Figure 21. Registradores

4. Considerações Finais

Este trabalho apresentou o projeto e a implementação de um processador de 32 bits com base na especificação RV32E do RISC-V, utilizando uma arquitetura de *pipeline* de cinco estágios. O objetivo principal foi criar um estudo de caso para o design de CPUs de baixo custo, implementando um subconjunto essencial de instruções para validar a funcionalidade básica do processador.

O desenvolvimento modular em Verilog HDL permitiu a construção e a análise detalhada de todos os componentes fundamentais do caminho de dados, incluindo a ULA, o banco de registradores e o sistema de memória, além das unidades dedicadas ao tratamento de *hazards*. A implementação alcançou sucesso parcial, com instruções como ADDI e LW funcionando corretamente na maioria dos cenários, o que valida a operação do caminho de dados principal.

No entanto, a fase de testes revelou desafios significativos, que são centrais para o aprendizado em projetos de arquitetura de computadores. O principal obstáculo identificado reside na lógica de controle do *pipeline*, especificamente no tratamento de desvios e na resolução de *hazards* de dados. Foi observado que a Unidade de *Forwarding*, embora presente, não cobre todos os cenários de dependência, exigindo a inserção manual de instruções NOP em alguns casos. Mais criticamente, as instruções de desvio condicional (BEQ, BNE, etc.) não funcionam como esperado: o cálculo do endereço de destino está incorreto e o mecanismo de *flush*, essencial para descartar instruções subsequentes, não está sendo acionado adequadamente.

Apesar dessas dificuldades, o projeto cumpriu seu propósito como uma ferramenta

educacional, fornecendo uma visão aprofundada sobre as complexidades do design de um processador *pipelined*. As falhas encontradas não diminuem o valor do trabalho, mas sim reforçam a importância de uma lógica de controle robusta e de testes exaustivos.

Os próximos passos, já delineados, focam na correção da unidade de controle de desvios, na implementação correta do *flush* e na revisão da Unidade de *Forwarding* para garantir a cobertura de todos os casos de dependência. Com essas melhorias, o processador tem o potencial de se tornar uma implementação completa e funcional do núcleo RV32E, servindo como uma base sólida para futuras otimizações e extensões.

References

- [1] Andrew Waterman, Yunsup Lee, David A. Patterson, Krste Asanović. *The RISC-V Instruction Set Manual, Volume I: User-Level ISA, Version 2.1*. RISC-V Foundation, 2019. Disponível em: <https://drive.google.com/file/d/1uviulnH-tScFfgrovvFCrj7Omv8tFtkp/view?pli=1>. Acesso em: 15 ago. 2025.
- [2] RISC-V International. *RISC-V Technical Specifications*. Disponível em: <https://lf-riscv.atlassian.net/wiki/spaces/HOME/pages/16154769/RISC-V+Technical+Specifications>. Acesso em: 15 ago. 2025.
- [3] RISC-V Foundation. *RISC-V Calling Convention*. Disponível em: <https://riscv.org/wp-content/uploads/2015/01/riscv-calling.pdf>. Acesso em: 15 ago. 2025.